

Job Market Analytics Dashboard

Full Project Documentation — Snakir Raza

1. What Is This Project?

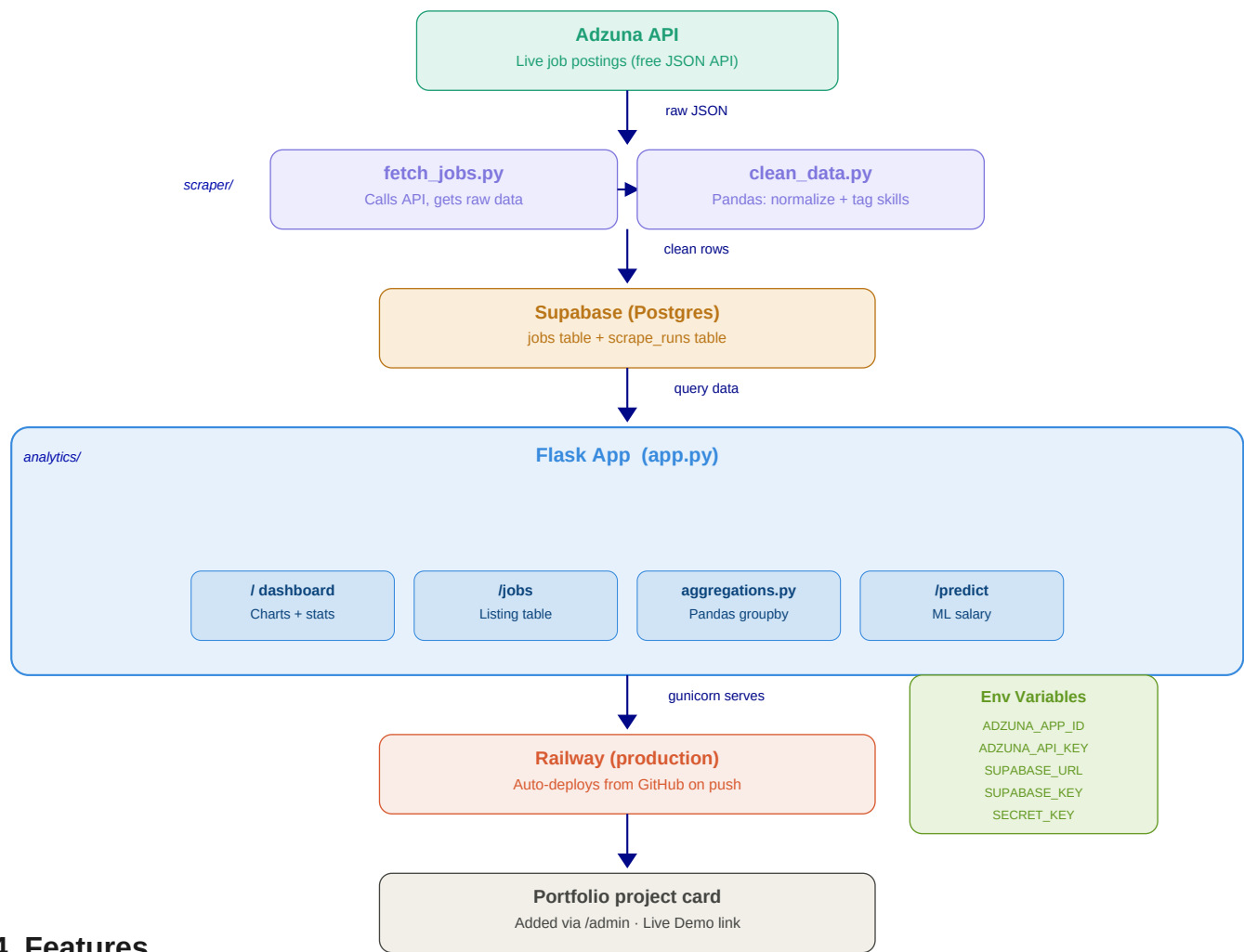
A full-stack web app that pulls live job postings from the Adzuna API, processes and stores them in Supabase, and presents an interactive analytics dashboard showing job market trends — most in-demand skills, top locations, salary ranges, and more. Deployed separately from the portfolio, then linked in as a project card.

2. Why This Project?

Reason	Detail
Combines all three skills	Full Stack (Flask) + Data Science (Pandas) + AI/ML (salary prediction)
Relevant background	Sales/BD experience means real understanding of the job market
End-to-end pipeline	Fetch → Clean → Store → Analyze → Visualize → Deploy
Real live data	Adzuna API — not a Kaggle CSV. Shows ability to handle messy data
Portfolio standout	Most student portfolios have no live analytics dashboard

3. Architecture Diagram

How data flows from the API all the way to the user's browser:



4. Features

MVP — build first

Feature	Description
Overview cards	Total jobs, jobs this week, top skill, avg salary
Skills demand chart	Bar chart of most requested skills across all postings
Location breakdown	Chart showing where jobs are geographically concentrated
Salary range chart	Distribution of salary data (where available from API)
Job listing table	Browsable grid of all jobs with links to original postings
Filters	Filter by category, location, and date range

Stretch Goals

Feature	Description
Salary prediction	Enter skills + location → predicted salary range (scikit-learn)
Trend over time	Line chart of skill demand week-over-week (scheduled scraping)
Skill gap tool	Select your skills → app shows which in-demand skills you're missing
CSV export	Download filtered job data as a spreadsheet

5. Tech Stack & Libraries

Layer	Technology	Why
Web framework	Flask	Same as portfolio — consistent stack
Data source	Adzuna API	Free official JSON API — no scraping risk
Data processing	Pandas	Industry standard for cleaning and EDA
Database	Supabase (Postgres)	Same as portfolio — one less new service
Charts	Plotly	Interactive charts from Pandas in one function call
ML (stretch)	scikit-learn	Simple salary regression on real data
Production server	Gunicorn	Required for Railway deployment
Deployment	Railway	Auto-deploys from GitHub on every push

6. File Structure

```

job-market-dashboard/
├── app.py Flask entry point. All routes defined here.
├── requirements.txt All Python dependencies – Railway installs from this.
├── runtime.txt python-3.11.9 – pins Python version for Railway.
├── Procfile web: gunicorn app:app – tells Railway how to start.
├── .env Secret API keys (gitignored – never pushed).
├── .gitignore Excludes: venv/ .env __pycache__
├── scraper/
│   ├── fetch_jobs.py Calls Adzuna API, returns raw job list.
│   ├── clean_data.py Pandas: normalizes fields, extracts skill tags.
│   └── analytics/
│       ├── aggregations.py Pandas groupby/count functions for charts.
│       └── ml_model.py (stretch) scikit-learn salary prediction.
├── static/css/style.css Dark theme CSS matching portfolio.
├── templates/
│   ├── base.html Shared layout (nav, footer, head).
│   ├── dashboard.html Main page with Plotly charts.
│   ├── jobs.html Job listing table with filters.
│   └── 404.html Custom error page.

```

7. Database Schema (Supabase — jobs table)

Column	Type	Description
id	uuid (PK)	Auto-generated unique ID
title	text	Job title
company	text	Company name
location	text	City / region
salary_min	numeric	Minimum salary (nullable)
salary_max	numeric	Maximum salary (nullable)
skills	text[]	Extracted skill tags e.g. ["Python", "Flask"]
category	text	Job category e.g. "IT Jobs", "Data Science"
source_url	text	Link to original job posting
posted_date	timestamp	When the job was posted
scraped_at	timestamp	When we fetched it from the API

8. Environment Variables

Variable	Where	What it does
ADZUNA_APP_ID	adzuna.com/developer	Identifies your app to Adzuna
ADZUNA_API_KEY	adzuna.com/developer	Authenticates API requests
SUPABASE_URL	Supabase → Settings → API	Your Supabase project URL
SUPABASE_KEY	Supabase → Settings → API	Supabase anon public key
SECRET_KEY	python secrets.token_hex(32)	Flask session security

9. Build Order

#	Step	Status
1	Project setup — venv, packages, file structure, local Flask server	✓ Done
2	Adzuna API — register, write fetch_jobs.py, test pulling data	Next
3	Data cleaning — Pandas, normalize fields, extract skill tags	
4	Supabase — create jobs table, store first batch of jobs	
5	Dashboard route — query Supabase, pass data to templates	
6	Plotly charts — skills bar chart, location chart, salary chart	
7	Jobs listing page — /jobs route + filters	
8	UI polish — dark CSS theme, responsive layout	
9	GitHub repo + Railway deployment	
10	Add to portfolio via /admin as project card	
11	(Stretch) Salary ML model + /predict route	

10. Key Lessons from Portfolio (Applied Here)

Problem in portfolio	Fix applied from day 1 here
Railway tried Python 3.13 → build failed	runtime.txt set to python-3.11.9

venv/ pushed to GitHub → massive repo	venv/ in .gitignore from day 1
.env committed accidentally	env in .gitignore from day 1
Supabase paused after inactivity → 500	Visit site weekly to keep Supabase active
Library imported but not in requirements.txt → crash	Add to requirements.txt immediately after pip install
Missing templates caused 500 errors	Create all templates before adding routes

Documentation: [Session 2](#) — Setup complete. Next: [Adzuna API connection](#).